

Sales Trend Analysis Using Aggregations

I performed a monthly sales trend analysis using SQL Server. I first imported a randomly generated sales dataset (CSV file) that contained 300 rows of transaction-level data, including order_id, order_date, amount, and product_id. After loading the data into SQL Server, I applied SQL aggregation functions to calculate the monthly revenue and monthly order volume.

I used YEAR(), MONTH(), SUM(), and COUNT(DISTINCT) to group the data by month and summarize total earnings and total orders for each month. Finally, I sorted the results to clearly analyze the sales performance across the year. This task demonstrates the use of grouping, ordering, and aggregate functions for time-based sales analysis in SQL Server.

Monthly Sales Trend Analysis Using SQL Aggregations (SQL Server)

```
use master
```

```
go
```

```
use online_sales
```

```
go
```

```
create database online_sales
```

```
go
```

--To find the schema of Database

```
sp_helpdb online_sales
```

```
go
```

```
select*from sales
```

```
go
```

--To find the schema of Table

```
sp_help sales
```

```
go
```

```
SELECT
```

```
    YEAR(order_date) AS [year],
```

```
    MONTH(order_date) AS [month],
```

```
    SUM(amount) AS monthly_revenue,
```

```
    COUNT(DISTINCT order_id) AS order_volume
```

```
FROM sales
```

```
GROUP BY
```

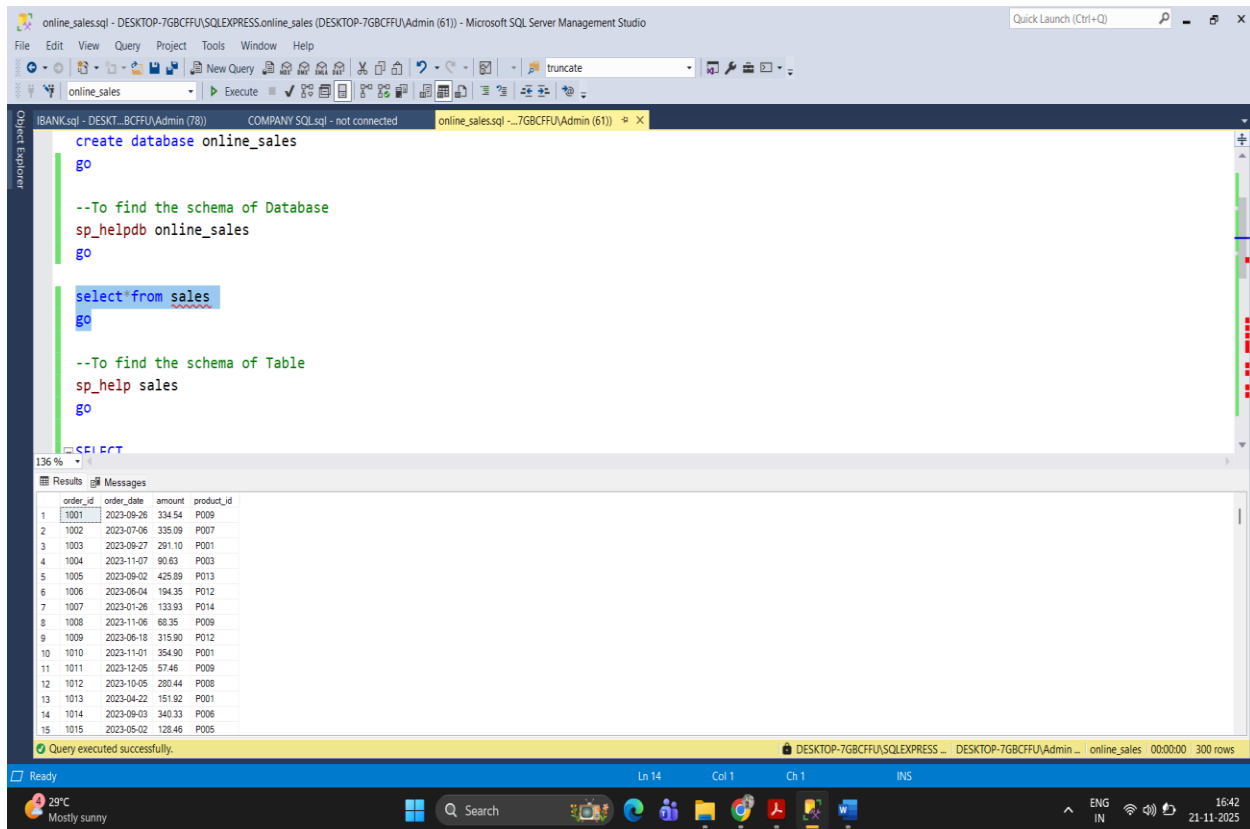
```
    YEAR(order_date),
```

MONTH(order_date)

ORDER BY

YEAR(order_date),

MONTH(order_date);



The screenshot displays the Microsoft SQL Server Management Studio interface. The main window shows a query editor with the following SQL code:

```
create database online_sales
go

--To find the schema of Database
sp_helpdb online_sales
go

select*from sales
go

--To find the schema of Table
sp_help sales
go
```

Below the query editor, the 'Results' pane shows the output of the 'select*from sales' query. The results are displayed in a table with the following columns: order_id, order_date, amount, and product_id. The table contains 15 rows of data.

order_id	order_date	amount	product_id
1001	2023-06-26	334.54	P009
1002	2023-07-06	335.09	P007
1003	2023-09-27	291.10	P001
1004	2023-11-07	90.63	P003
1005	2023-09-02	425.89	P013
1006	2023-06-04	194.35	P012
1007	2023-01-26	133.93	P014
1008	2023-11-06	68.35	P009
1009	2023-06-18	315.90	P012
1010	2023-11-01	354.90	P001
1011	2023-12-05	57.46	P009
1012	2023-10-05	280.44	P008
1013	2023-04-22	151.92	P001
1014	2023-09-03	340.33	P006
1015	2023-05-02	128.46	P005

The status bar at the bottom indicates that the query was executed successfully, returning 300 rows.

online_sales.sql - DESKTOP-7GBCFU\SQLEXPRESS.online_sales (DESKTOP-7GBCFU\Admin (61)) - Microsoft SQL Server Management Studio

File Edit View Query Project Tools Window Help

Execute | New Query | truncate

Object Explorer | IBANK.sql - DESKTOP_...BCFFU\Admin (78) | COMPANY SQL.sql - not connected | online_sales.sql - ...7GBCFU\Admin (61) | X

```
go
SELECT
    YEAR(order_date) AS [year],
    MONTH(order_date) AS [month],
    SUM(amount) AS monthly_revenue,
    COUNT(DISTINCT order_id) AS order_volume
FROM sales
GROUP BY
    YEAR(order_date),
    MONTH(order_date)
ORDER BY
    YEAR(order_date),
    MONTH(order_date);
```

136 %

Results Messages

	year	month	monthly_revenue	order_volume
1	2023	1	10014.93	32
2	2023	2	7768.44	23
3	2023	3	7070.31	29
4	2023	4	4509.42	17
5	2023	5	7624.08	30
6	2023	6	5738.35	24
7	2023	7	5715.71	22
8	2023	8	7061.55	27
9	2023	9	10950.92	36
10	2023	10	4592.76	16
11	2023	11	7780.73	29
12	2023	12	3807.24	15

Query executed successfully.

DESKTOP-7GBCFU\SQLEXPRESS ... | DESKTOP-7GBCFU\Admin ... | online_sales | 00:00:00 | 12 rows

Ready | Ln 27 | Col 9 | Ch 9 | INS

29°C Mostly sunny | Search | ENG IN | 16:36 21-11-2025